SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA,MANGALURU - 574146



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
GROUP PROJECT ON

" PRACTICAL APPLICATIONS OF BIG DATA, MACHINE LEARNING, DEEP LEARNING, AND CHATBOT "

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING

Submitted By,

ANANYA DHAGE	**01SU25AI012**
BHAVANI	**01SU25AI022**
AISHWARYA BJ	**01SU25AI010**
JYOTHI	**01SU25AI033**

UNDER THE GUIDANCE OF

PROF. MAHESH KUMAR VB

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING ENGINEERING
2025-2026

CONTENTS

Module 2 BIG DATA PROCESS MAPPING 2

2.1 Introduction and Problem Statement 2

2.2 Data Sources 2

2.3 Data Ingestion and Storage 3

2.4 Processing Layer 3

2.5 Output Layer and Feedback Loop 3

Module 3 BUILD A SIMPLE ML PROCESS FLOW 4

3.1 Introduction and Problem Definition 4

3.2 Data Collection and Preparation 4

3.3 Feature Extraction and Algorithm Selection 5

3.4 Training and Validation 5

3.5 Testing, Evaluation and Deployment 5

Module 4 DEEP LEARNING APPLICATION PITCH 6

4.1 Introduction and Problem Statement 6

4.2 Required Data 6

4.3 Network Type and Architecture 6

4.4 Hardware Needs and Deployment 7

4.5 Ethical Concerns and Conclusion 7

Module 5 CHATBOT BUILD & DEMO 8

5.1 Introduction and Objective 8

5.2 Chatbot Design and Platform 8

5.3 Conversation Logic and Flow 8

5.4 Sample User Interactions 9

5.5 Limitations, Improvements and Conclusion 9

5.6 References 9

MODULE 2
BIG DATA PROCESS MAPPING

Question (Module 2): *Big Data Process Mapping: Groups select a real-world big data system (like Google Maps, Amazon recommendations, or smart city sensors) and map out the entire data flow: data sources, storage, processing, and output.*

2.1 Introduction and Problem Statement

Big data refers to information that is too large, too fast, or too varied to be captured, stored, and analysed with a traditional single-machine database. It is usually described through the five Vs: volume, the quantity of records; velocity, the speed at which data arrives; variety, the mixture of tables, logs, text, images, and click streams; veracity, the trustworthiness of noisy or incomplete data; and value, the insight that justifies the cost of collecting and processing it. Mapping a big data process means tracing data from the moment it is generated to the moment it influences a decision shown to a user, and naming every storage layer, system, and algorithm that it passes through on the way.

The group selected an online-shopping recommendation system, of the kind used by large e-commerce platforms such as Amazon, as its case study. A recommendation engine is a good example because it converts billions of small user actions, such as searches, clicks, views, purchases, and ratings, into a short personalised list of products that appears within a fraction of a second. The problem statement adopted was how raw behavioural and catalogue data travels from the shopper's device through ingestion, storage, and processing to a personalised suggestion on screen, and how the shopper's reaction then feeds back into the system. The data flow was organised under four headings: data sources, ingestion and storage, processing, and output, with a feedback loop closing the cycle. The map presented here is a simplified, educational model built from publicly available descriptions; the internal architecture of a commercial platform is proprietary, so the stages should be read as a representative pipeline rather than an official blueprint.

Representative data-flow map of an e-commerce recommendation system (simplified, educational model)

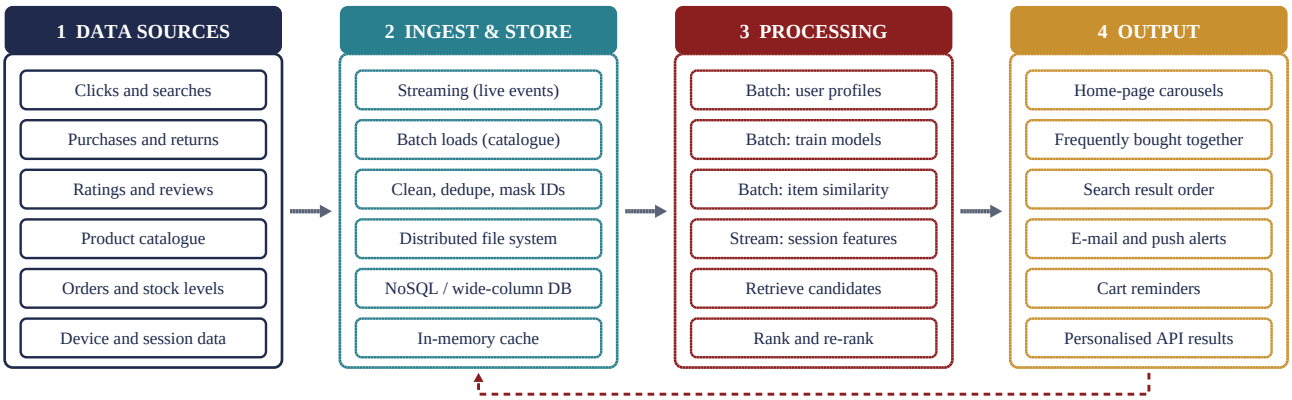


Figure 2.1: *End-to-end data-flow map of the case study, from data sources to user-facing output, with the feedback loop that returns every output to the system as new input.*

2.2 Data Sources

The first layer contains the places where data originates. The most valuable source is behavioural data from shoppers: product searches, page views, clicks, items added to a cart or wish list, purchases, returns, ratings, and written reviews. Each event carries a user or session identifier, a product identifier, a timestamp, and the device or channel used. The second source is the product catalogue, which holds titles, descriptions, categories, brands, prices, images, and seller details, and which changes whenever sellers add or update listings. The third group is transactional and operational data such as orders, payments, stock levels, and delivery status, which tells the system whether a recommended item can actually be bought and delivered.

Additional context comes from external and derived sources: seasonal calendars and festival dates, location and language settings, advertising and campaign data, and the text of customer reviews and questions. Together these sources show the variety dimension of big data, because a single recommendation may depend on tables, free text, images, time series, and event streams at once.

2.3 Data Ingestion and Storage

The ingestion stage must receive data reliably however it arrives. Slowly changing data such as catalogue updates and order histories is loaded in batches at regular intervals, while live events such as clicks and searches are published to a messaging and stream-processing layer that absorbs sudden bursts of traffic during sales, removes duplicate events, attaches consistent timestamps, and delivers each event to the systems that need it. Tools of this kind are typified by distributed log systems such as Apache Kafka and stream engines such as Spark Streaming or Flink. Records are also validated here: malformed events are rejected, bots and obvious spam are filtered out, and personal identifiers are masked or replaced by pseudonymous keys so that privacy obligations are respected.

The storage layer then keeps each kind of data in the form that suits how it will be read. Raw event logs and images are stored cheaply in a distributed file system or object store, which spreads every file across many machines and replicates it against hardware failure. Structured records such as the product catalogue and order history are kept in scalable databases, including relational stores for transactions and wide-column or key-value (NoSQL) stores such as Cassandra or DynamoDB for fast lookups by user or product. Hot data, such as a shopper's most recent clicks or a precomputed list of popular items, is held in in-memory caches such as Redis so that it can be read in milliseconds, and a data warehouse or data lake keeps the cleaned history for analysis and model training.

2.4 Processing Layer

Processing turns raw data into knowledge and runs in two cooperating modes. Batch processing periodically scans the full stored history, often overnight, to prepare the long-term knowledge of the system. Distributed frameworks such as Hadoop MapReduce or Apache Spark clean the data, build user and product profiles, compute item-to-item similarity, and train recommendation models. A classic technique is collaborative filtering, which finds products that are viewed or bought by similar groups of customers, the familiar idea behind customers who bought this also bought that. Content-based filtering compares product attributes with a shopper's past choices, and modern systems blend both with deep-learning models that learn compact numerical representations, called embeddings, of users and items.

Stream processing runs continuously over the live data. As a shopper browses, recent clicks are grouped by session, updated in a real-time feature store, and combined with the long-term profile so that recommendations react within seconds to a change of interest. When a page is requested, a candidate-generation step first narrows millions of products to a few hundred likely items, and a ranking model then scores those candidates using features such as predicted relevance, price, popularity, availability, and freshness. A final re-ranking step applies business rules, removes out-of-stock or already purchased items, and adds diversity so that the list is not filled with near-identical products.

2.5 Output Layer and Feedback Loop

The output layer is everything the shopper sees. Recommendations appear as carousels such as recommended for you and frequently bought together, as personalised home-page rows, as the ordering of search results, as e-mail and push-notification suggestions, and as reminders about items left in the cart. These outputs reach web browsers, mobile apps, and partner websites through application programming interfaces and must be returned in well under a second, so precomputed lists and caches are used wherever possible and the heavy computation is done in advance.

The process map is a loop rather than a straight line. Every impression, click, purchase, skip, or return is logged and sent back to the ingestion stage, where it becomes new training data. Online experiments called A/B tests compare a new model with the current one on real shoppers, using measures such as click-through rate and conversion rate, and only the better model is rolled out. Because every output becomes an input, the system improves as more people use it, a property often called the data network effect. The main challenges noted by the group were customer privacy and consent, the cold-start problem for new users and new products with little history, bias towards already popular items, filter bubbles that narrow what shoppers see, and the cost of running large clusters. Table 2.1 summarises how the five Vs appear in this case study.

Table 2.1: The five Vs mapped to the case study

V	Meaning	Where it appears in the case study
Volume	Quantity of records	Billions of clicks, orders, ratings, product records, and images
Velocity	Speed of arrival	Continuous click and search events; sharp bursts during sale periods
Variety	Mixture of formats	Tables, review text, images, event logs, and session data
Veracity	Trustworthiness of noisy data	Bots, accidental clicks, and fake reviews are filtered or down-weighted
Value	Insight that justifies the cost	Relevant suggestions, higher conversion, and a better shopping experience

MODULE 3
BUILD A SIMPLE ML PROCESS FLOW

Question (Module 3): *Build a Simple ML Process Flow: Groups create a complete flowchart for a machine learning project, covering data collection, feature extraction, algorithm selection, training, testing, and evaluation.*

3.1 Introduction and Problem Definition

Machine learning builds programs that improve at a task by learning patterns from data instead of following hand-written rules. A machine learning project is therefore a disciplined sequence of stages, each producing an output that feeds the next, and each able to send the team back to an earlier stage when the results disappoint. The group's task was to draw this sequence as a complete flowchart covering data collection, feature extraction, algorithm selection, training, testing, and evaluation, and to explain every symbol used in it.

To keep the chart concrete, a familiar supervised problem was used as the running example: predicting the selling price of a house from its attributes, such as built-up area, number of bedrooms, age of the building, and distance from the city centre. The target is a continuous number rather than a yes-or-no label, so this is a regression problem. The flowchart starts with a terminator symbol and a process box that defines the problem and the success criteria, for example that the predicted price must be clearly better than a baseline that always guesses the average price. It then uses rectangles for processing steps, parallelograms for data input and output, diamonds for decisions, dashed arrows for iteration loops, and solid arrows for the direction of flow.

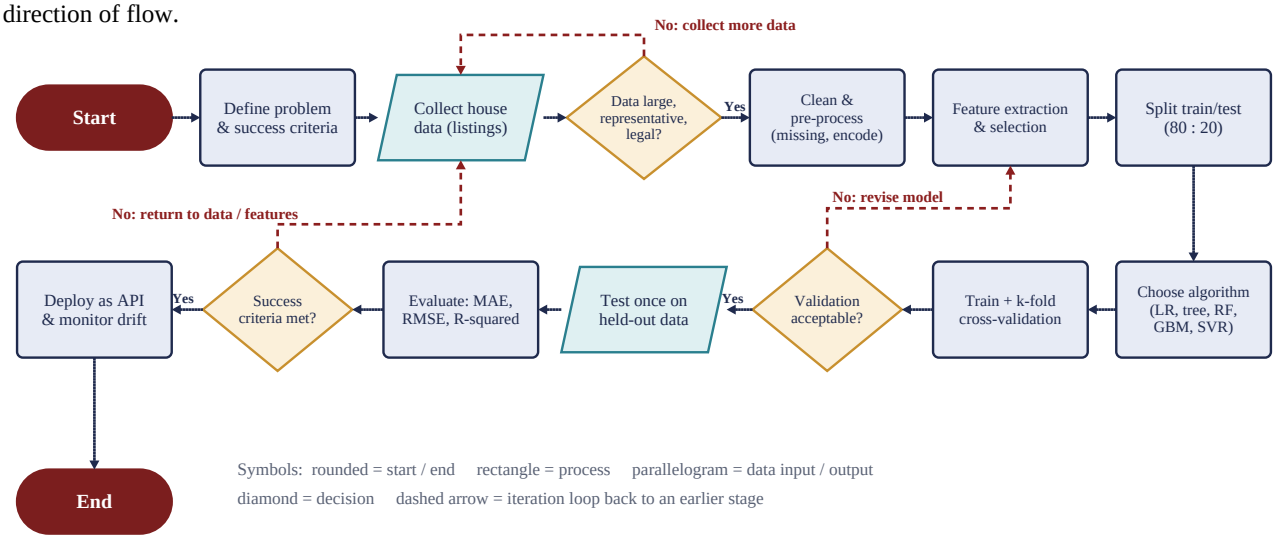


Figure 3.1: *Complete machine learning process flowchart for the house-price prediction example. Dashed arrows are the iteration loops back to earlier stages.*

3.2 Data Collection and Preparation

The first working stage is data collection. For the house-price example the team gathers listing and registration records that describe each property, including area, number of rooms, age, floor, location, nearby amenities, and the final sale price, which is the label the model must learn to predict. Public datasets or local records may be used, and the records are joined on a property identifier so that each row describes exactly one house. At this point a decision diamond asks whether the data is large enough, representative of the houses the model will meet in future, and legally and ethically permitted for this use; if not, the arrow loops back and more data is collected.

Raw data is rarely ready for learning, so the next box is cleaning and preprocessing. Missing values are removed or filled with the median, the most frequent category, or a predicted value; duplicate rows are dropped; impossible entries such as a negative area are corrected; and outliers are examined rather than deleted blindly, because a luxury villa may be unusual but genuine. Categorical fields such as locality are converted to numbers by one-hot encoding, numeric fields are standardised so that columns with large units do not dominate, and a heavily skewed target such as price is often transformed with a logarithm so that errors on cheap and expensive houses are treated more evenly.

3.3 Feature Extraction and Algorithm Selection

Feature extraction derives meaningful inputs from the cleaned data, because raw columns are seldom the best predictors. The team engineers features such as the age of the house, calculated as the current year minus the year built, the total number of rooms, the price per square foot of nearby sales, and the distance to the nearest school, hospital, or bus stop. A feature selection box then removes correlated or uninformative columns using correlation analysis, model-based importance scores, or principal component analysis, which shortens training and lowers the risk of overfitting. The data is finally split into a training portion of about eighty per cent and a held-out test portion that stays untouched until the end. Scaling statistics are computed on the training portion only, so that no information leaks from the test data.

Algorithm selection is drawn as a decision, because the right choice depends on the type and size of the data, the need for interpretability, and the available computing power. For tabular regression the candidates are linear regression, a fast and interpretable baseline; decision trees, which give readable rules; random forests, which average many trees to reduce variance; gradient-boosted trees, which often achieve the best accuracy on structured data; and support vector regression or small neural networks for strongly non-linear patterns. Table 3.1 summarises the trade-offs.

Table 3.1: Candidate algorithms for tabular regression

Algorithm	Main strength	Main limitation
Linear regression	Fast, interpretable baseline	Limited to mostly linear patterns
Decision tree	Readable if-then rules	Unstable; easily overfits
Random forest	Averages many trees, lowers variance	Slower; harder to explain
Gradient-boosted trees	Often best accuracy on structured data	Needs careful tuning
SVR / small neural network	Captures strongly non-linear patterns	Less interpretable; scaling needed

3.4 Training and Validation

In training, the chosen algorithm is fitted to the training data: it searches for the parameters, such as regression coefficients or tree split points, that minimise a loss function measuring the gap between predicted and true prices. A common loss for regression is the mean squared error. Hyperparameters such as tree depth, the number of trees, or the learning rate are set by the designer rather than learned, and to choose them without touching the test set the team uses k-fold cross-validation, in which the training data is divided into k parts and the model is trained k times, each time validating on a different part.

The flowchart contains a decision loop at this point. If both the training and validation errors are high, the model is underfitting, and the team returns to feature engineering or picks a more expressive algorithm. If the training error is low but the validation error is much higher, the model is overfitting, and the team adds regularisation such as Ridge or Lasso penalties, simplifies the model, collects more data, or removes features. Learning curves, which plot error against the amount of training data, help to tell the two situations apart. Only when validation performance is acceptable does the arrow leave the loop and continue to the testing stage.

3.5 Testing, Evaluation and Deployment

Testing is the single, final measurement of the chosen model on the held-out data, which simulates the unseen houses it will meet in production. For regression the team calculates the mean absolute error, the root mean squared error, and the coefficient of determination, known as R-squared, and also inspects residual plots to see whether errors are random or systematically larger for certain price ranges or localities. Table 3.2 lists the metrics and their meaning.

Evaluation also inspects feature importance and the worst errors, and compares the result with the baseline and with the success criteria written at the start. The final decision diamond asks whether those criteria are met. If they are, the model is packaged behind an application programming interface for deployment and monitored for data drift, since house prices change with the market and the model must be retrained when its errors begin to grow; if not, an arrow returns to data collection or feature extraction.

Table 3.2: Regression metrics used in the evaluation

Metric	Formula in words	Reading
MAE	Average of |actual price - predicted price|	Typical error in the same unit as the price
RMSE	Square root of the average squared error	Penalises large mistakes more heavily
R-squared	1 - (unexplained variation / total variation)	Share of price variation explained by the model

MODULE 4
DEEP LEARNING APPLICATION PITCH

Question (Module 4): *Deep Learning Application Pitch: Each group proposes a novel DL application (e.g., energy optimization, earthquake detection), detailing required data, network type, hardware needs, and ethical concerns.*

4.1 Introduction and Problem Statement

Deep learning is the branch of machine learning that uses neural networks with many layers to learn features directly from raw data such as images, signals, and text, instead of relying on hand-crafted features. Its strength is greatest when data is large, complex, and high-dimensional, which is exactly the situation in many environmental and safety problems. The group's task was to propose a novel deep learning application and to detail the data, the network type, the hardware, and the ethical concerns involved. The application proposed is a rainfall-triggered landslide and flash-flood early warning system for the Western Ghats and coastal Karnataka, built on a hybrid convolutional and recurrent neural network.

The problem is real and local. Heavy monsoon rainfall on steep, saturated slopes regularly causes landslides, flooded roads, and damaged homes along the Karnataka coast and the Ghat sections that connect it to the interior. Current warnings rely largely on fixed rainfall thresholds and regional forecasts, which cannot capture how terrain, vegetation, soil saturation, and several days of accumulated rain interact at a particular slope. The novelty of the proposal lies in fusing three different data types, satellite imagery, rainfall time series, and ground-sensor readings, inside one network that outputs a risk level for each small area together with an estimated lead time. This is a design proposal for a pitch; the figures and performance targets mentioned are goals to be validated, not results already achieved.

4.2 Required Data

The system needs three families of data. Image data includes optical and radar satellite scenes, such as freely available Sentinel-2 and Sentinel-1 imagery, together with a digital elevation model from which slope, aspect, and curvature are derived, and land-use and vegetation maps. Radar imagery is especially useful because it can see through the monsoon clouds that block optical sensors. Time-series data includes rainfall from gauges and satellite estimates, forecasts from the national meteorological service, temperature, and humidity. Ground-sensor data includes soil moisture, pore-water pressure, and slope tilt measured at instrumented sites, which show how saturated a slope really is.

Labels are the hardest part. Historical landslide and flood inventories from geological and disaster-management agencies, verified news reports, and field surveys provide the positive examples, while quiet periods and stable slopes provide the negatives. Because dangerous events are rare, the dataset is heavily imbalanced, so the team plans to use weighted or focal loss, careful resampling, and evaluation measures such as recall and precision instead of plain accuracy. All layers must be georeferenced, aligned to a common grid, cloud-masked, and normalised, and the split into training, validation, and test sets should be made by time period and by region so that the model is judged on seasons and places it has never seen.

Table 4.1: Data plan for the proposed application

Data type	Possible source	Role in the model
Satellite imagery	Optical and radar scenes	Land cover, bare soil, and slope surface
Terrain model	Digital elevation data	Slope, drainage, and curvature of each cell
Rainfall and weather	Rain gauges, satellite estimates, forecasts	Short- and long-term rain accumulation
Ground sensors	Soil-moisture and tilt probes	Real-time saturation and ground movement
Event labels	Landslide and flood inventories	Ground truth for supervised training

4.3 Network Type and Architecture

The proposed network is a hybrid. A convolutional neural network, or CNN, processes the image and terrain patches: its convolution layers act as learned filters that detect edges, scars, bare soil, drainage lines, and vegetation patterns, and pooling layers shrink the maps while keeping the strongest responses. A long short-term memory network, or LSTM, processes the rainfall and sensor sequences. Its gated cells decide what to remember and what to forget, which lets it capture the fact that several days of steady rain matter as much as one heavy burst. The two encoders produce feature vectors that are concatenated in a fusion layer, passed through dense layers with ReLU activations and dropout, and finished with a softmax output that gives the probability of low, medium, or high risk. Figure 4.1 shows the architecture.

Training would use the Adam optimiser with a weighted cross-entropy loss, early stopping on the validation set, and transfer learning, in which a CNN pre-trained on a large image collection is fine-tuned on the satellite patches so that fewer labelled events are needed. The reason for choosing a deep network rather than a simple threshold or a classical model is that the inputs are raw images and long sequences whose useful patterns are difficult to describe by hand, and the network can learn them from examples. Success is judged mainly by recall, because a missed event is costly, balanced against the false-alarm rate and the lead time given to residents.

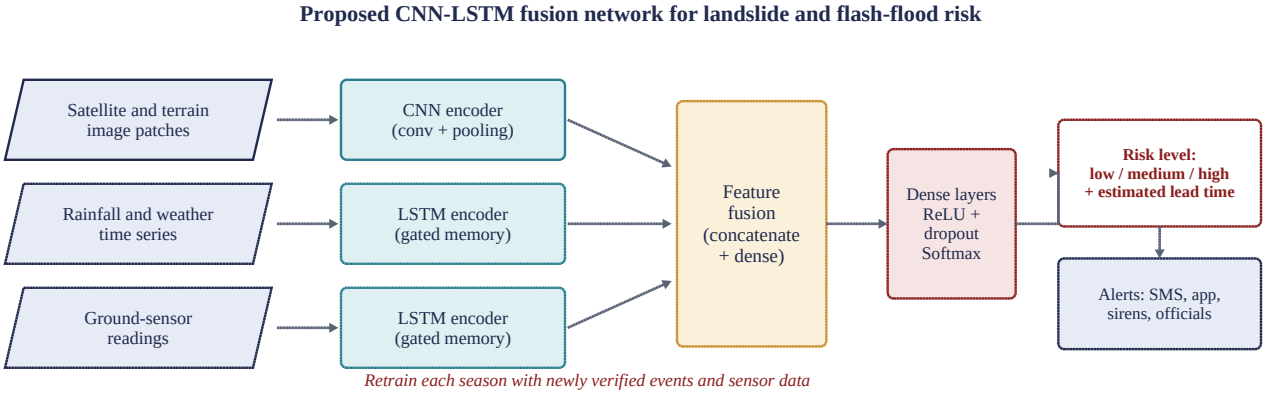


Figure 4.1: *Proposed hybrid network: a CNN encodes imagery, LSTMs encode time series, and a fusion layer produces a risk level and lead time.*

4.4 Hardware Needs and Deployment

Hardware needs differ between training and deployment. Training the hybrid network on large stacks of satellite imagery needs a GPU workstation or a cloud GPU instance with ample video memory, plenty of system memory, and fast storage for hundreds of gigabytes of image tiles; frameworks such as TensorFlow or PyTorch handle the computation. Once trained, the model is compressed by pruning and quantisation so that inference is light enough to run on low-power edge devices of the Jetson or Raspberry Pi class installed at sensor stations in the hills. These stations are powered by solar panels and report over GSM or low-power radio links, because mains power and network coverage are unreliable in monsoon conditions.

A central server collects station outputs and fresh satellite scenes, runs the full model for each grid cell, and sends alerts to disaster-management officials, local authorities, and residents through SMS, a mobile application, and community sirens. The cloud side is also used to retrain the model each season with newly verified events. Because heavy rain is exactly when failures are most likely, the design includes redundant communication paths and regular sensor maintenance.

Table 4.2: Hardware plan

Stage	Hardware	Reason
Training	GPU workstation or cloud GPU, large storage	Heavy image and sequence computation
Edge inference	Jetson or Raspberry Pi class, solar power	Low-power, on-site predictions during outages
Communication	GSM or low-power radio, SMS gateway	Reliable alerts from remote hills

4.5 Ethical Concerns and Conclusion

Ethical concerns are central to a safety application. The most important is the balance between false alarms and missed alarms: too many false alarms cause alarm fatigue and loss of trust, while a missed event can cost lives, so thresholds must be set with disaster experts and communicated honestly. Fairness is a second concern, because remote villages have fewer sensors and less historical data, which could make the model least accurate precisely where communities are most vulnerable; coverage and accuracy should therefore be audited by region. Privacy and consent matter when sensors are placed on private land or when mobile alert lists are built, and such data must be stored securely and used only for warnings.

Transparency and accountability are also required. Techniques such as saliency maps and feature-importance scores can show officials why a cell was marked high risk, and the final decision to evacuate should always stay with accountable human authorities, with the model acting only as decision support. The environmental cost of training large models and the effect of climate change on rainfall patterns, which can make past data less reliable, should also be monitored. In conclusion, the proposal shows how deep learning can turn heterogeneous data into timely local warnings, provided that it is built with careful data practices, honest evaluation, and human oversight.

MODULE 5
CHATBOT BUILD & DEMO

Question (Module 5): *Chatbot Build & Demo: Groups create a simple rule-based chatbot (e.g., with Dialogflow). Present its logic, user interactions, and improvements.*

5.1 Introduction and Objective

A chatbot is a software program that converses with users through text or voice. Chatbots broadly fall into two groups. A rule-based chatbot follows predefined rules: it recognises what the user wants through patterns or trained example phrases and replies with answers written in advance. A generative chatbot, in contrast, composes new sentences using a large language model. Rule-based bots are predictable, inexpensive, and easy to control, which makes them well suited to focused tasks such as answering frequently asked questions, while their weakness is that they cannot handle topics outside the rules written for them.

The objective of this module was to design a simple rule-based chatbot with Dialogflow, Google's platform for building conversational agents, and to present its logic, sample user interactions, and possible improvements. The chatbot chosen is a campus helpdesk assistant for prospective students, which answers common questions about admission, courses, fees, hostel facilities, and contact details at any hour. The intended learning was to practise conversational design, to understand how intents, training phrases, and entities work together, and to see how the limits of rules lead naturally to improvements. The behaviour described below is the design of the demonstration bot, and the answers it gives are placeholders that an administrator would replace with the institution's official information.

5.2 Chatbot Design and Platform

Dialogflow organises a chatbot as an agent that is made of intents. An intent represents one thing the user wants, such as asking about admission, and it contains training phrases, which are example sentences a user might type, together with the response the bot should give. Dialogflow uses natural language understanding to match a new message to the closest intent even when the wording differs from the examples. Entities extract useful values from the sentence: a system entity can capture dates or numbers, and a custom entity such as a department entity can store a list of departments with their synonyms so that different spellings point to the same value. Contexts let the bot remember what was just discussed so that a short follow-up such as what about the fees is understood, and a fulfillment webhook can connect the bot to a database or another service when a fixed reply is not enough.

The design followed a simple sequence. First the group listed the questions that students ask most often and grouped them into intents: Default Welcome, Admission Process, Course Information, Fee Enquiry, Hostel Information, Contact Us, Thank You and Goodbye, and Default Fallback. Second, eight to ten training phrases of different wording and length were written for each intent. Third, a clear and friendly response was written for each intent, with quick-reply suggestion chips so that users can tap instead of typing. Fourth, required parameters and follow-up contexts were added where a question needs more detail, for example asking which department the user means. Finally the agent was tested in the Dialogflow simulator and published through the web demo integration, which can also be extended to messaging platforms.

5.3 Conversation Logic and Flow

The logic of the chatbot is deterministic and can be drawn as the flow in Figure 5.1. The user sends a message, and the agent compares it with the training phrases of every intent and calculates a match confidence. If the confidence is above the threshold, the intent is considered matched and the agent extracts any entities in the sentence, checks the active context, and selects the response written for that intent. If a required detail is missing, the agent asks a short prompting question and waits for the answer before replying, a process called slot filling. If no intent reaches the threshold, the Default Fallback intent apologises, asks the user to rephrase, and offers a menu of topics the bot can handle, after which the conversation continues from the user's next message.

Because every answer comes from a rule or a stored response, the same question always produces the same reply, which makes the bot easy to test and to audit. The rules also include a polite handover message that gives the office telephone number and e-mail address when a question needs a human, so that users are never left at a dead end. Greetings and thanks are handled by their own small intents so that the conversation feels natural without leading the user away from the main purpose.

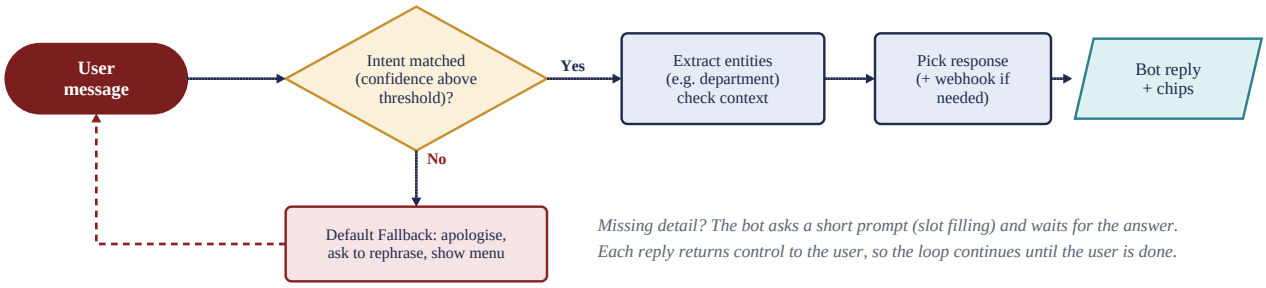


Figure 5.1: *Conversation logic of the rule-based campus helpdesk chatbot, including the fallback loop.*

5.4 Sample User Interactions

Table 5.1 shows representative interactions with the chatbot. The test approach is to try every intent with phrases that were not used in training, with different wording, spelling mistakes, mixed upper and lower case, and short fragments, and to check whether the right intent is matched and the right response is shown. Questions deliberately placed outside the bot's scope are used to check that the fallback behaves politely. Each wrongly matched or unmatched phrase is recorded and then added to the correct intent as a new training phrase, so that the agent improves with each round of testing.

Table 5.1: Representative user interactions and the intent matched

User message	Intent matched	Bot behaviour
Hi, is anyone there?	Default Welcome	Greets the user and offers topic chips: Admission, Courses, Fees, Hostel
How do I apply for B.Tech?	Admission Process	Explains the application steps and asks if the user wants course details
Tell me about the AI and ML department	Course Information	Reads the department value and returns the matching course summary
What about the fees?	Fee Enquiry (follow-up)	Uses the saved context and gives the fee information
Do you provide hostel?	Hostel Information	Describes the hostel facilities and how to book a room
Can I play cricket there?	Default Fallback	Apologises, asks to rephrase, and shows the topic menu

5.5 Limitations, Improvements and Conclusion

The bot has clear limitations. It can only answer what has been written into its intents, it may confuse two intents that have similar wording, it handles one question at a time, and it keeps only a short memory of the conversation. Its answers do not update by themselves when fees or dates change. Language is another limit, since the first version supports English only, although most prospective students would be more comfortable in Kannada or Hindi.

Several improvements are planned. More training phrases should be gathered from real conversations, and a log of unmatched questions should be reviewed regularly. A webhook can connect the bot to live data such as admission status or event dates. Multilingual support and voice input would make it more accessible, and a feedback button after each answer would measure user satisfaction. A live-agent handover can be added for complex cases, and, for open-ended questions, a generative model that retrieves answers only from approved college documents could be combined with the rules, keeping control over accuracy. In conclusion, the four modules together follow one idea from the data upward: big data supplies the raw material, machine learning and deep learning find patterns in it, and conversational AI delivers the result to people in a form they can use, with responsible design at every stage.

5.6 References

1. Marz, N. and Warren, J., Big Data: Principles and Best Practices of Scalable Real-Time Data Systems. Manning, 2015.
2. Ricci, F., Rokach, L., and Shapira, B. (eds.), Recommender Systems Handbook. Springer, 2015.
3. Geron, A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media, 2022.
4. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning. MIT Press, 2016.
5. Hochreiter, S. and Schmidhuber, J., Long Short-Term Memory. Neural Computation, 9(8), 1997.
6. Jurafsky, D. and Martin, J. H., Speech and Language Processing, 3rd ed. draft. Stanford University.
7. Google Cloud, Dialogflow documentation: agents, intents, entities, and contexts. https://cloud.google.com/dialogflow/docs